Milestone 2.1: Approximate Bayesian Inference

INFO 521 · Machine Learning Foundations · project milestone brief · From Inference to Discovery · Gate A (Supervised) · 1 of 2

How to read this
This is the assignment brief for one project milestone. The Satisfactory criteria below are authoritative: every one must be met, and it is all-or-nothing rather than partial credit. A Not-Yet returns with targeted feedback and one revise cycle. Do the work in the notebook distributed through Classroom 50; this PDF is the brief, not the workspace.

Course-schedule placement. Part 2 of the project: begins at the midpoint, due Week 7.5. This milestone covers M5 and is due Week 6. Gated by Checkpoint quiz 2.1 (Newton-Raphson update, Wk 5); Gate A.

Where conjugacy ends

In milestone 1.3 a Gaussian prior met a Gaussian-noise likelihood and the posterior was *itself* Gaussian, closed form, no sampling. That was a gift of **conjugacy**. Part 2 opens by taking the gift away: we **binarize systolic BP into a hypertension label** and ask a Bayesian **logistic regression** to predict it from the non-BP features. The Bernoulli-likelihood-meets-Gaussian-prior pair is *not* conjugate, the posterior has no closed form, so you must *approximate* it. You will do this three ways and line them up against the three lenses of Part 1.

Ground rules. Implement every estimator from scratch in NumPy/SciPy. Do **not** use scikit-learn or any off-the-shelf solver, sampler, or optimizer. The `info521` library gives you data, plotting, and self-checks only, never a fitting routine.

From scratch rules out libraries, never your own prior work: your Unit 5 `logistic`, `newton_map`, `laplace_covariance`, and `MetropolisSampler` are the certified starting points for §§2 to 5. Carry them in and adapt them (more features, the τ^2 prior, the unpenalized intercept), and note in one line what you changed. The homework certified the methods; this milestone grades the clinical application.

Leakage rule (read once, obey everywhere in Gate A). The label is *derived from blood pressure* (`hypertension(ds)` applies SBP ≥ 130 or DBP ≥ 80). The features are therefore the **six non-BP columns** `ds["X"]` (`age, bmi, waist, chol, hdl, hba1c`). **Never** put `sbp` or `dbp`, or any function of them, into the feature matrix: that would leak the label into its own predictors and inflate every score. `load_clinical()` already excludes both BP columns; keep it that way.

What “Satisfactory” means for 2.1

You reach Satisfactory when **all** of the following hold:

- ☐ You **state the model** (Gaussian prior on $\mathbf{w}$, Bernoulli likelihood with a sigmoid mean) and explain in prose **why the posterior is non-conjugate**, with an explicit callback to the closed-form Gaussian posterior you relied on in 1.3.
- ☐ You implement the **log-posterior, its gradient, and its Hessian** from scratch and **verify the gradient** against your loss with `checks.expect_gradient` (PASS).
- ☐ You implement **Newton–Raphson (IRLS)** from scratch and reach the **MAP** estimate $\mathbf{w}_{\text{MAP}}$.
- ☐ You form the **Laplace approximation**: a Gaussian posterior centred at the MAP with covariance from the Hessian.
- ☐ You implement a **Metropolis sampler** from scratch (proposal, acceptance ratio, burn-in) and show **trace plots** and **posterior summaries**.
- ☐ You **compare** the three treatments (MAP point, Laplace Gaussian, Metropolis samples) and name the explicit **parallel to Part 1’s point / likelihood / posterior** lenses.
- ☐ **Play artifact** (below) complete and interpreted.
- ☐ **Checkpoint quiz 2.1 (Newton–Raphson update)** passed.

Setup

```
import numpy as np
from info521 import data, plotting, checks

plotting.use_course_style()
ds = data.load_clinical()

X_raw = ds["X"]                        # (n, 6): age, bmi, waist, chol, hdl, hba1c
feature_names = ds["features"]
labels, rule = data.hypertension(ds)  # leakage-safe ACC/AHA label in {0, 1}

print(f"{X_raw.shape[0]} patients · {X_raw.shape[1]} features = {feature_names}")
print(f"label = {rule}")
print(f"hypertension prevalence = {labels.mean():.3f}")
```

Notation follows PML: $\sigma(a) = 1/(1 + e^{-a})$ is the logistic sigmoid, $\mu_n = \sigma(\mathbf{w}^\top \mathbf{x}_n)$ is the predicted probability, and $y_n \in \{0, 1\}$ is the hypertension label.

0 · Data prep: split and standardize (no leakage)

Before any modelling: hold out a test set, and **standardize features using statistics estimated on the training set only**. Computing the mean/SD on the full data (or on the test set) leaks test information into training, a subtle but real form of leakage you must avoid here and in 2.2.

```
def train_test_split(X, y, test_frac=0.25, seed=521):
    # TODO: shuffle indices with a seeded RNG and split into train/test.
    # Return X_train, X_test, y_train, y_test.
    raise NotImplementedError

def standardize_fit(X_train):
    # TODO: return (mean, sd) computed on TRAIN ONLY.
```

```
    raise NotImplementedError

def standardize_apply(X, mean, sd):
    # TODO: return (X - mean) / sd.
    raise NotImplementedError

# TODO: split, fit the scaler on train, apply to train AND test, then prepend an
# intercept column of ones to each design matrix. Keep the intercept UNpenalized
# by the prior below (or note explicitly if you choose to penalize it).
```

1 · The model, and why it is non-conjugate

In a markdown cell, write the model and argue non-conjugacy:

- **Prior.** $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{S}_0)$, with $\mathbf{S}_0 = \tau^2 \mathbf{I}$ (the prior variance τ^2 is the knob you sweep in the play artifact).
- **Likelihood.** $y_n \mid \mathbf{x}_n, \mathbf{w} \sim \text{Bernoulli}\big(\sigma(\mathbf{w}^\top \mathbf{x}_n)\big)$, points conditionally independent.
- **Why no closed form.** Multiply the Gaussian prior by the Bernoulli likelihood: the sigmoid sits *inside* an exponential of a sum of $\log \sigma$ terms, so the product is **not** the kernel of any standard distribution, unlike the Gaussian × Gaussian product in 1.3, which stayed Gaussian. State this contrast explicitly: *what* about 1.3 made the posterior closed-form, and *what* breaks here.

2 · Log-posterior, gradient, Hessian (from scratch)

Work with the **negative** log-posterior (an energy to *minimize*), $E(\mathbf{w}) = -\log p(\mathbf{w} \mid \mathcal{D}) + \text{const}$. Derive its gradient $\mathbf{g}(\mathbf{w}) = \nabla E$ and Hessian $\mathbf{H}(\mathbf{w}) = \nabla^2 E$ by hand (this derivation is **Checkpoint quiz 2.1**), then implement all three. Verify the gradient against your own loss **before** trusting it downstream.

```
def sigmoid(a):
    # TODO: numerically stable logistic sigmoid.
    raise NotImplementedError

def neg_log_posterior(w, X, y, tau2):
    # TODO: negative log prior + negative log Bernoulli likelihood (a scalar).
    # Watch numerical stability near mu -> 0 or 1.
    raise NotImplementedError

def grad_neg_log_posterior(w, X, y, tau2):
    # TODO: analytic gradient of E(w). Shape == w.
    raise NotImplementedError

def hess_neg_log_posterior(w, X, y, tau2):
    # TODO: analytic Hessian of E(w). Shape == (d, d).
    raise NotImplementedError

# Verify the gradient with the finite-difference checker (ships no analytic
# gradient: it differentiates YOUR loss numerically). Bind the data/tau2 so the
# checked functions are pure functions of w:
#
# f  = lambda w: neg_log_posterior(w, X_tr, y_tr, tau2)
```

```
# gf = lambda w: grad_neg_log_posterior(w, X_tr, y_tr, tau2)
# checks.expect_gradient(f, gf, w0, name="neg-log-posterior gradient")
```

3 · Newton–Raphson (IRLS) to the MAP

The MAP minimizes E. Newton–Raphson iterates $\mathbf{w} \leftarrow \mathbf{w} - \mathbf{H}(\mathbf{w})^{-1}\mathbf{g}(\mathbf{w})$ until convergence, which for logistic regression is exactly **iteratively reweighted least squares**. Solve the linear system; do **not** form an explicit inverse.

```
def fit_map_newton(X, y, tau2, w0=None, n_iter=50, tol=1e-8):
    # TODO: Newton-Raphson loop using your grad/Hessian. Your Unit 5
    # newton_map adapts directly; note what changed. Use np.linalg.solve on
    # H w_step = g. Track and return the convergence history so you can plot it.
    raise NotImplementedError

# TODO: fit on the training split; report iterations to convergence; sanity-check
# that the gradient norm at the solution is ~0.
```

4 · Laplace approximation

A second-order Taylor expansion of E at the mode gives a Gaussian posterior $q(\mathbf{w}) = \mathcal{N}\!\left(\mathbf{w}_{\mathrm{MAP}}, \mathbf{H}(\mathbf{w}_{\mathrm{MAP}})^{-1}\right)$: the **negative Hessian of the log-posterior** (= the Hessian of E) at the mode is the posterior **precision**.

```
def laplace_posterior(X, y, tau2, w_map):
    # TODO: return (mean=w_map, cov = inverse of the Hessian at w_map).
    raise NotImplementedError

# TODO: report the posterior SD of each weight (sqrt of the cov diagonal) and say
# which features the model is most / least certain about.
```

5 · Metropolis sampler (from scratch)

Sample the posterior directly with random-walk Metropolis. Propose $\mathbf{w}' = \mathbf{w} + \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, s^2\mathbf{I})$; accept with probability $\min\!\left(1, \exp[-E(\mathbf{w}') + E(\mathbf{w})]\right)$ (work in **log space**). Discard a burn-in, then summarize.

```
def metropolis(X, y, tau2, w_init, n_samples=20000, proposal_sd=0.1,
               burn_in=2000, seed=521):
    # TODO: random-walk Metropolis. Your Unit 5 MetropolisSampler adapts
    # directly; note what changed. Compute the acceptance ratio with your
    # neg_log_posterior (log space!). Return the post-burn-in chain AND the
    # empirical acceptance rate.
    raise NotImplementedError

# TODO: run it from w_MAP; report the acceptance rate; plot a trace per weight with
# plotting helpers; report posterior means/SDs from the samples.
```

A healthy chain mixes (the trace looks like a “fuzzy caterpillar”, not a slow drift or a flat line) and accepts roughly 20–50% of proposals. If yours doesn’t, the play artifact is where you diagnose it.

6 · Three treatments, one posterior

Put the three side by side for a single weight (or a 2-D pair):

Treatment	What you get	Part 1 parallel
MAP (Newton)	a **point** $\mathbf{w}_{\text{MAP}}$	the OLS/MLE **point estimate** (1.1)
Laplace	a **Gaussian** around the mode	the MLE **+ error bars** (1.2)
Metropolis	**samples** from the true posterior	the full **Bayesian posterior** (1.3)

```
# TODO: overlay, for one weight, the MAP point, the Laplace Gaussian density, and a
# histogram of the Metropolis samples. Where do Laplace and Metropolis agree, and
# where does the Gaussian approximation distort the true (possibly skewed) posterior?
```

Then **write 150–250 words** naming the parallel explicitly: how is "point / likelihood-with-uncertainty / full posterior" the *same* progression you walked in Part 1, and what is genuinely new here (the posterior is *approximated*, not exact)?

Play artifact: prior variance & proposal scale

Two knobs, two stories:

1. **Prior variance τ^2.** Sweep it across several orders of magnitude. Show the effect on $\mathbf{w}_{\text{MAP}}$ and on the posterior width (Laplace SDs and Metropolis spread). Connect to 1.3: a tight prior (small τ^2) shrinks weights toward zero; a vague prior lets the data speak.
2. **Metropolis proposal scale s.** Sweep it. Show the effect on the **acceptance rate** and on **mixing**, too small $\Rightarrow$ high acceptance but a chain that crawls; too large $\Rightarrow$ most proposals rejected and the chain sticks. Show a **trace plot** and a **running-mean** plot for each scale.

Write 150–250 words tying the two sweeps together: how prior strength and sampler tuning each shape what you conclude about the posterior.

```
# TODO: the two sweeps + trace/running-mean diagnostics + written interpretation.
```

> **i** Gate
>
> This milestone is not Satisfactory until **Checkpoint quiz 2.1 (derive the Newton–Raphson update for the MAP of Bayesian logistic regression: gradient, Hessian, update)** is passed. The checkpoint is an open-book D2L quiz, due the week before this milestone; the auto-graded portion resolves on submission and written responses allow one revision.

Looking ahead

In **2.2** you keep the same train/test split and the same leakage-safe features, but swap the *probabilistic* classifier for a **margin-based** one, a from-scratch soft-margin SVM, and ask which framing wins, and what each actually expresses: a calibrated probability versus a hard boundary.